SE 3242: Android Application Development

Week 2: Kotlin Essentials

Session 2: Classes in Kotlin (constructors, inheritance, interfaces, visibility modifiers, data classes)

Engr. Daniel MOUNE

moune.daniel@ictuniversity.edu.cm

ICT University, Cameroon Campus, Yaoundé
P.O. Box 526, 1 Avenue Dispensaire Messassi, Zoatoupsi

Imparting ICTs in all academic disciplines

February 24, 2026

Quick Recap – What We Learned Last Session

Functions

- Function syntax, parameters, return types
- Expression body vs block body
- Default and named arguments
- Varargs, infix functions
- Extension functions

Collections & Lambdas

- Lists, Sets, Maps (immutable vs mutable)
- Iteration: `for`, `forEach`
- Higher-order functions: `map`, `filter`, `fold`
- Function types and lambdas (`it`)

Kotlin vs Dart

We saw many parallels – now we dive into object-oriented programming.

"From functions to objects: let's model real-world entities!"

Today's Roadmap – Object-Oriented Programming in Kotlin

Part 1: Classes & Constructors (30 min)

- Class syntax, properties
- Primary and secondary constructors
- Initializer blocks

Part 2: Inheritance (30 min)

- Open classes and methods
- Overriding rules
- Abstract classes

Part 3: Interfaces (25 min)

- Defining interfaces
- Multiple interface implementation
- Default implementations

Part 4: Data Classes & Sealed Classes (35 min)

- `data class` – auto-generated methods
- Sealed classes for restricted hierarchies

Part 5: Project Milestone 3 (10 min)

"Today we build the foundation for robust app architecture."

Classes in Kotlin – The Basics

Class Declaration

```
class Person {
    var name: String = ""
    var age: Int = 0
}
```

Properties are declared directly in the class body.

Creating an Instance

```
val person = Person()
person.name = "Alice"
person.age = 25
println("${person.name} is ${person.age} years old")
```

No `new` keyword – just call the constructor like a function.

"Classes encapsulate data and behavior."

Primary Constructor – Properties in One Place

Primary Constructor Syntax

Properties can be declared directly in the constructor:

```
class Person(val name: String, var age: Int)
```

`val` makes it read-only (getter only), `var` makes it mutable (getter and setter).

Usage

```
val person = Person("Alice", 25)
println(person.name)    // Alice
person.age = 26         // OK because var
```

Default values & named arguments

```
class Person(
    val name: String,
    var age: Int = 0
)
val p = Person(name = "Bob")
```

"The primary constructor makes class definitions extremely concise."

Initializer Blocks – Code That Runs During Construction

`init` Blocks

Executed when an instance is created, after primary constructor parameters are available.

```
class Person(val name: String, var age: Int) {
    init {
        require(age >= 0) { "Age must be non-negative" }
        println("Person $name created")
    }
}
```

Multiple init blocks

They run in the order they appear in the class body.

```
class Person(val name: String) {
    init { println("First init") }
    init { println("Second init") }
}
```

"Use `init` for validation or logging – runs before secondary constructors."

Secondary Constructors – When You Need More Flexibility

Declaring secondary constructors

Use `constructor` keyword. Must delegate to primary constructor (if present) using `this`.

```
class Person(val name: String, var age: Int) {
    constructor(name: String) : this(name, 0) {
        println("Secondary constructor called")
    }
}
```

Usage

```
val p1 = Person("Alice", 25)   // primary
val p2 = Person("Bob")
// secondary (age = 0)
```

Without primary constructor

```
class Student {
    var name: String
    constructor(name: String) { this.name =
}
```

"Prefer default values or factory functions over secondary constructors."

Inheritance – Extending Classes

Open classes

Classes are `final` by default – mark with `open` to allow inheritance.

```
open class Animal(val name: String) {
    open fun makeSound() {
        println("Some generic sound")
    }
}
```

Derived class

Use `:` and call superclass constructor. Override functions with `override`.

```
class Dog(name: String) : Animal(name) {
    override fun makeSound() {
        println("$name says Woof!")
    }
}
```

"`override` is mandatory – helps avoid accidental overrides."

Overriding Properties and Using `super`

Overriding properties

A property can be overridden if it is open in the superclass.

```
open class Animal(open val sound: String = "?")
class Dog : Animal() {
    override val sound = "Woof"
}
```

Calling superclass implementation

Use super to access members of the parent.

```
open class Animal {
    open fun makeSound() = println("Animal sound")
}
class Dog : Animal() {
    override fun makeSound() {
        super.makeSound()
        println("Dog adds its own sound")
    }
}
```

Abstract Classes – Incomplete by Design

Abstract class

Cannot be instantiated. May contain abstract members (no implementation) and concrete members.

```
abstract class Shape {
    abstract fun area(): Double
    fun description() = "This is a shape"
}
```

Implementing an abstract class

```
class Circle(val radius: Double) : Shape() {
    override fun area():
        Double = Math.PI * radius * radius
}
```

Abstract properties

```
abstract class Vehicle {
    abstract val wheels: Int
}
class Car : Vehicle() {
    override val wheels = 4
}
```

"Abstract classes define contracts – perfect for framework design."

Interfaces – Defining Behavior

Interface declaration

Interfaces define abstract methods and properties (without state).

```
interface Drivable {
    fun drive()  // abstract method
    val maxSpeed: Int  // abstract property
}
```

Implementing an interface

```
class Car(override val maxSpeed: Int)
: Drivable {
    override fun drive() {
        println(
        "Driving at max $maxSpeed km/h"
                        )
    }
}
```

Multiple interfaces

A class can implement several interfaces.

```
interface Flyable {
    fun fly()
}
class FlyingCar : Drivable, Flyable {
    override fun drive() { ... }
    override fun fly() { ... }
}
```

"Interfaces separate what from how – key to polymorphism."

Default Methods and Conflicts

Interfaces with default implementations

Use `default` keyword? No – just provide a body.

```
interface Drivable {
    fun drive() { println("Driving...") }  // default
}
```

Conflict resolution (multiple inheritance)

When two interfaces have the same method, you must override.

```
interface A { fun foo() { println("A") } }
interface B { fun foo() { println("B") } }
class C : A, B {
    override fun foo() {
        super<A>.foo()  // call A's version
        super<B>.foo()  // call B's version
    }
}
```

Use `super<InterfaceName>.method()` to disambiguate.

Data Classes – Classes That Hold Data

`data class` benefits

Automatically generates `toString()`, `equals()`, `hashCode()`, `copy()`, and component functions.

```
data class User(val name: String, val age: Int)
```

toString

```
val u = User("Alice", 25)
println(u)
// User(name=Alice, age=25)
```

copy

```
val older = u.copy(age = 26)
```

Destructuring

```
val (name, age) = u
println("$name is $age")
```

"Perfect for model classes – reduces boilerplate dramatically."

Sealed Classes – Defining a Closed Set of Subclasses

What are sealed classes?

All subclasses are known at compile time – they must be in the same file. Perfect for representing state (e.g., loading, success, error).

```
sealed class Result {
    data class Success(val data: String) : Result()
    data class Error(val message: String) : Result()
    object Loading : Result()
}
```

Exhaustive when handling

```
fun handle(result: Result) = when (result) {
    is Result.Success -> println("Data: ${result.data}")
    is Result.Error -> println("Error: ${result.message}")
    Result.Loading -> println("Loading...")
} // No else needed     all cases covered
```

"Sealed classes bring pattern matching-like safety to Kotlin."

Object Declarations – Singletons and Companions

Object declaration (singleton)

Creates a class with exactly one instance.

```
object AppConfig {
    val version = "1.0"
    fun show() = println("App version $version")
}
// Usage: AppConfig.show()
```

Companion object (static-like members)

```
class MyClass {
    companion object {
        const val TAG = "MyClass"
        fun create(): MyClass = MyClass()
    }
}
MyClass.TAG
MyClass.create()
```

Factory pattern with companion

```
class User private constructor(val name: Str
    companion object {
        fun createGuest() = User("Guest")
    }
}
```

"No `static` keyword – objects and companions fill the gap."

Visibility Modifiers – Who Can See What?

Kotlin's access modifiers

`public` (default)	visible everywhere
`internal`	visible inside the same module
`protected`	visible in subclasses
`private`	visible inside the class/file

Example

```
open class Parent {
    val a = 1              // public
    internal val b = 2     // internal
    protected val c = 3    // protected
    private val d = 4      // private
}
class Child : Parent() {
    fun test() {
        println(a)    // OK
        println(b)    // OK (same module)
        println(c)    // OK (protected)
        // println(d) // ERROR: private
    }
}
```

Top-level visibility

Top-level functions/classes can be `public`, `internal`, or `private`.

```
// file: utils.kt
private fun helper() { ... }
internal val version = "1.0"
```

"Encapsulation is key – use the least permissive visibility possible."

Nested vs Inner Classes – Static by Default

Nested class (static)

A class inside another class without an explicit reference to the outer class.

```
class Outer {
    private val msg = "Hello"
    class Nested {
        fun show() = "Nested" // cannot access msg
    }
}
// Usage: Outer.Nested().show()
```

Inner class (non-static)

Marked with `inner` – holds a reference to the outer instance.

```
class Outer {
    private val msg = "Hello"
    inner class Inner {
        fun show() = msg + " from Inner"
        fun getOuter() = this@Outer
    }
}
```

Properties with Custom Accessors

Custom getter

Compute a value on the fly instead of storing it.

```
class Rectangle(val width: Int, val height: Int) {
    val area: Int
        get() = width * height   // computed each time
}
```

Custom setter

Add validation or side effects.

```
class Person {
    var age: Int = 0
        set(value) {
            if (value >= 0) field = value
            else throw IllegalArgumentException()
        }
}
```

Property without backing field

```
val isAdult: Boolean
    get() = age >= 18
// no backing field
```

Delayed Initialization – `lateinit` and `lazy`

`lateinit` – non-null var initialized later

For dependency injection or Android views.

```
class MyActivity {
    lateinit var repository: Repository
    fun setup() {
        repository = Repository()
    }
    fun use() { repository.doSomething() }
}
```

Access before init throws `UninitializedPropertyAccessException`.

`lazy` – thread-safe by default

For `val` properties initialized on first access.

```
val expensive: String by lazy {
    println("Computing...")
    "Hello"
}
```

Delegation – Reuse Without Inheritance

Class delegation

Implement an interface by delegating to another object.

```
interface SoundBehavior { fun makeSound() }
class ScreamBehavior : SoundBehavior {
    override fun makeSound() = println("AAARGH!")
}
class Person(sound: SoundBehavior) : SoundBehavior by sound
```

Person automatically forwards makeSound calls to sound.

Property delegation

Reusable property logic (e.g., lazy, observable).

```
import kotlin.properties.Delegates

var name: String by Delegates.observable("initial") {
    prop, old, new -> println("$old -> $new")
}
```

Kotlin vs Dart – Classes and Constructors

Kotlin

```kotlin
class Person(
    val name: String,
    var age: Int = 0
) {
    init { println("Person created") }
}
// Usage:
val p = Person("Alice", 25)
```

Dart

```dart
class Person {
  final String name;
  int age;
  Person(this.name, {this.age = 0});
}
// Usage:
var p = Person("Alice", age: 25);
```

Observations

- Kotlin primary constructor is part of class header; Dart uses constructor syntax in body.
- Both support default values (Kotlin: =, Dart: = in named parameters).
- Kotlin's `init` blocks vs Dart's initializer list or constructor body.

Kotlin vs Dart – Inheritance and Interfaces

Kotlin

```kotlin
open class Animal(open val name: String)
interface Flyable { fun fly() }

class Bird(name: String) : Animal(name),
    Flyable {
    override fun fly() =
        println("$name flies")
}
```

Dart

```dart
class Animal {
  String name;
  Animal(this.name);
}
abstract class Flyable { void fly(); }

class Bird extends Animal implements
    Flyable {
  Bird(String name) : super(name);
  @override void fly() => print('$name
      flies');
}
```

Key Differences

- Kotlin requires `open` for extensible classes; Dart classes are extensible by default.
- Kotlin uses `:` for both inheritance and implementation; Dart uses `extends` and `implements` separately.
- Both support abstract methods and interfaces.

Kotlin vs Dart – Data Classes and Value Objects

Kotlin (data class)

```kotlin
data class User(
    val id: Int,
    val name: String
)
// Auto-generates equals, hashCode,
// toString, copy
```

Dart (manual or @immutable)

```dart
class User {
  final int id;
  final String name;
  User(this.id, this.name);
  @override bool operator==(Object other)
      => ...
  @override int get hashCode => ...
}
//Or use equatable package.
```

Takeaway

Kotlin's `data class` eliminates boilerplate; Dart requires manual implementation or external packages.

"Data classes are one of Kotlin's biggest productivity boosters."

Exercise 1: Model a Zoo

Task

Create a class hierarchy for animals:

- Abstract class `Animal` with `name` and abstract `makeSound()`.
- Two concrete subclasses: `Dog` and `Cat`, each with their own sound.
- Add a property `legs` to `Animal` (override appropriately).
- Create a list of animals, iterate and print each sound.

Expected output

```
Buddy says Woof!
Whiskers says Meow!
```

"Practice inheritance, abstract classes, and polymorphism."

Exercise 2: Model Network Request State with Sealed Class

Task

Define a sealed class `NetworkState` representing:

- `Loading`
- `Success(data: String)`
- `Error(message: String)`

Write a function `handleState(state: NetworkState)` that prints appropriate messages.

Example usage

```
val states = listOf(
    NetworkState.Loading,
    NetworkState.Success("User data loaded"),
    NetworkState.Error("Network timeout")
)
states.forEach { handleState(it) }
```

"Sealed classes make state handling exhaustive and safe."

Exercise 3: Drawable Shapes with Interfaces

Task

Define an interface `Drawable` with a function `draw()`. Create classes `Circle` and `Square` that implement `Drawable`. Each should have appropriate properties (radius, side length) and print a simple ASCII representation.

Sample implementation idea

```
interface Drawable {
    fun draw()
}

class Circle(val radius: Int) : Drawable {
    override fun draw() {
        println("  ***  ")
        println(" *   * ")
        println("  ***  ")
    }
}
```

Project Milestone 3: Object-Oriented Domain Model

Objective

Refine your app's data model using object-oriented principles. Build a small class hierarchy that represents the core entities of your app.

Requirements

- Create at least **three classes** with proper inheritance or interface implementation.
- Use an **abstract class** or **interface** to define common behavior.
- Include at least one **data class**.
- Override `toString()` in at least one class (or rely on data class).
- Demonstrate **polymorphism** by storing different subclass instances in a collection and calling overridden methods.
- Write a short `main()` that showcases your hierarchy.

Submission

Due: **Next Monday 23:59**

Submit a single Kotlin file named `Milestone3_YourName.kt` on LMS.

"This milestone ensures you can design with OOP in mind."

Summary – What We Covered Today

Classes & Objects

- Class syntax, primary/secondary constructors
- `init` blocks, properties
- Nested vs inner classes

Inheritance & Polymorphism

- Open classes and methods
- Overriding rules, abstract classes
- Interfaces and multiple inheritance

Advanced Features

- Data classes – `toString`, `copy`, destructuring
- Sealed classes for restricted hierarchies
- Object declarations, companion objects
- Visibility modifiers, delegation

"You now have the tools to build well-structured, maintainable Kotlin code."

Coming Up: Functional Kotlin & Coroutines

Next Week – Session 1

We'll dive into more advanced functional programming concepts and asynchronous programming:

- Scope functions: `let`, `apply`, `run`, `with`, `also`
- Higher-order functions revisited
- Introduction to coroutines: lightweight concurrency
- Suspend functions and structured concurrency

Why this matters

Coroutines are essential for Android development – they simplify background tasks, network calls, and database operations without blocking the UI thread.

"Get ready to make your apps responsive and efficient!"

Questions & Discussion

Any questions on today's topics?

Reminders

- Project Milestone 3 due next Monday
- Office hours: Wed 14:00–16:00, Fri 10:00–12:00
- Practice exercises available on LMS

"See you next week for coroutines!"